

Rapport de Projet

Marketing Mix Modeling et Attribution Multi-Touch

Modélisation Bayésienne · Pipeline Data · Visualisation BI

Réalisé par

Hamza Elmourabit · Saad Benhaimer

Encadré par : Pr. Hrimch

**Pipeline ETL · Feature Engineering · Modèle
Bayésien -
Dashboard Streamlit · Looker BI**

Résumé Exécutif

Ce document constitue le rapport technique de référence du projet de **Marketing Mix Modeling (MMM)**. Il détaille l'ensemble de la chaîne de valeur, depuis l'ingestion des données brutes jusqu'à la restitution opérationnelle via des tableaux de bord interactifs.

Le projet repose sur quatre piliers fondamentaux :

1. **Pipeline Data** — validation, nettoyage, enrichissement événementiel, feature engineering (adstock, saturation, lags, interactions) et normalisation ;
2. **Modélisation** — estimation des contributions canal, élasticités et simulation budgétaire via un modèle de référence Ridge et une approche Bayésienne cible (PyMC) ;
3. **Visualisation** — dashboard Streamlit multi-pages et intégration de dashboards Looker embarqués ;
4. **Industrialisation** — export BigQuery, schéma analytique, configuration centralisée et exécution automatisée.

Le système produit un jeu de données de **52+ variables** prêtes pour la modélisation, incluant les transformations MMM critiques (adstock géométrique, saturation de Hill, lags temporels et interactions inter-canaux), ainsi que les variables de contrôle (saisonnalité, événements, tendance).

Objectifs du projet

- Mesurer l'efficacité de chaque canal marketing
- Estimer les contributions relatives (attribution multi-touch)
- Simuler l'impact de différentes allocations budgétaires
- Fournir une interface de visualisation opérationnelle

Table des matières

Résumé Exécutif	1
1 Introduction	4
1.1 Contexte	4
1.2 Objectifs du Projet	4
1.3 Périmètre	4
2 Architecture du Système	5
2.1 Vue d'Ensemble	5
2.2 Flux de Données Global	5
2.3 Composants Détaillés	5
3 Pipeline de Données	8
3.1 Vue d'Ensemble	8
3.2 Étape 1 : Validation Robuste	8
3.3 Étape 2 : Nettoyage Intelligent	8
3.4 Étape 3 : Feature Engineering Avancé	9
3.4.1 Adstock Géométrique	9
3.4.2 Saturation — Transformation de Hill	9
3.4.3 Lags et Interactions	9
3.4.4 Résumé des Features Créées	9
3.5 Étape 4 : Normalisation	11
3.6 Étape 5 : Export BigQuery	11
4 Modélisation MMM	13
4.1 Objectifs	13
4.2 Modèle de Référence (Ridge)	13
4.3 Approche Bayésienne (Cible)	13
4.4 Spécification du Modèle Bayésien	14
4.5 Attribution Multi-Touch	14
5 Visualisation et BI	16
5.1 Architecture de Visualisation	16
5.2 Pages du Dashboard Streamlit	16
5.2.1 Analyse des Canaux	16

5.2.2	Scénarios Budgétaires (What-If)	16
5.3	Intégration Looker	16
6	Déploiement et Exécution	20
6.1	Prérequis	20
6.2	Installation	20
6.3	Stack Technologique	20
6.4	Variables de Configuration	20
6.5	Exécution	20
6.6	Artefacts de Production	21
7	Résultats et Indicateurs de Performance	23
7.1	Objectifs de Performance	23
7.2	Résultats Attendus	23
8	Perspectives et Développements Futurs	25
8.1	Court Terme (1-3 mois)	25
8.2	Moyen Terme (3-6 mois)	25
8.3	Long Terme (6-12 mois)	25
9	Conclusion	26
A	Structure des Répertoires	27
B	Exemple de Code PyMC	29
C	Modèles LookML	31

Chapitre 1

Introduction

1.1 Contexte

Dans un environnement marketing de plus en plus fragmenté, la mesure de l'efficacité publicitaire est un enjeu stratégique. Les méthodes d'attribution *last-click* ou *multi-touch* basées sur les cookies sont remises en question par les restrictions de confidentialité (iOS 14+, fin des cookies tiers).

Le **Marketing Mix Modeling** offre une alternative robuste, agrégée et respectueuse de la vie privée.

1.2 Objectifs du Projet

Le projet vise à délivrer une solution *end-to-end* permettant de :

- Transformer les données brutes marketing et ventes en features exploitables pour la modélisation
- Estimer la contribution marginale de chaque canal publicitaire au chiffre d'affaires
- Simuler des allocations budgétaires optimisées
- Fournir des interfaces de visualisation opérationnelles (Streamlit + Looker)

1.3 Périmètre

Le périmètre couvre :

- **9 canaux marketing** : Google Search, Shopping, PMax, Display, Video, Meta Facebook, Instagram, Other, TikTok
- **Variable cible** : Revenus de premières ventes
- **Enrichissement événementiel** : 17+ jours spéciaux (Black Friday, Noël, etc.)
- **Historique** : Données journalières sur 1-2 ans

Chapitre 2

Architecture du Système

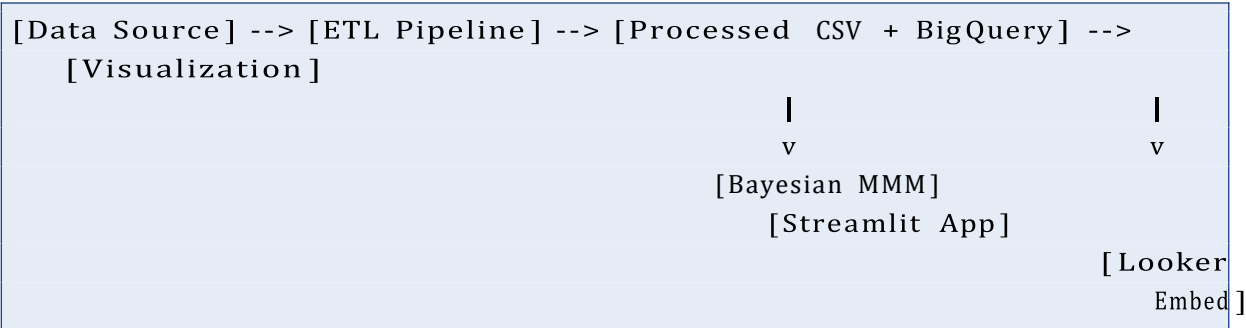
2.1 Vue d’Ensemble

L’architecture est organisée en trois couches principales : l’ingestion et la transformation (ETL), la modélisation analytique, et la visualisation BI. Le flux part des données brutes, traverse la pipeline de traitement, alimente l’entrepôt BigQuery, puis nourrit les modèles et les dashboards.

2.2 Flux de Données Global

Le flux de données suit un processus séquentiel et automatisé :

Listing 2.1 – Flux de données global



2.3 Composants Détaillés

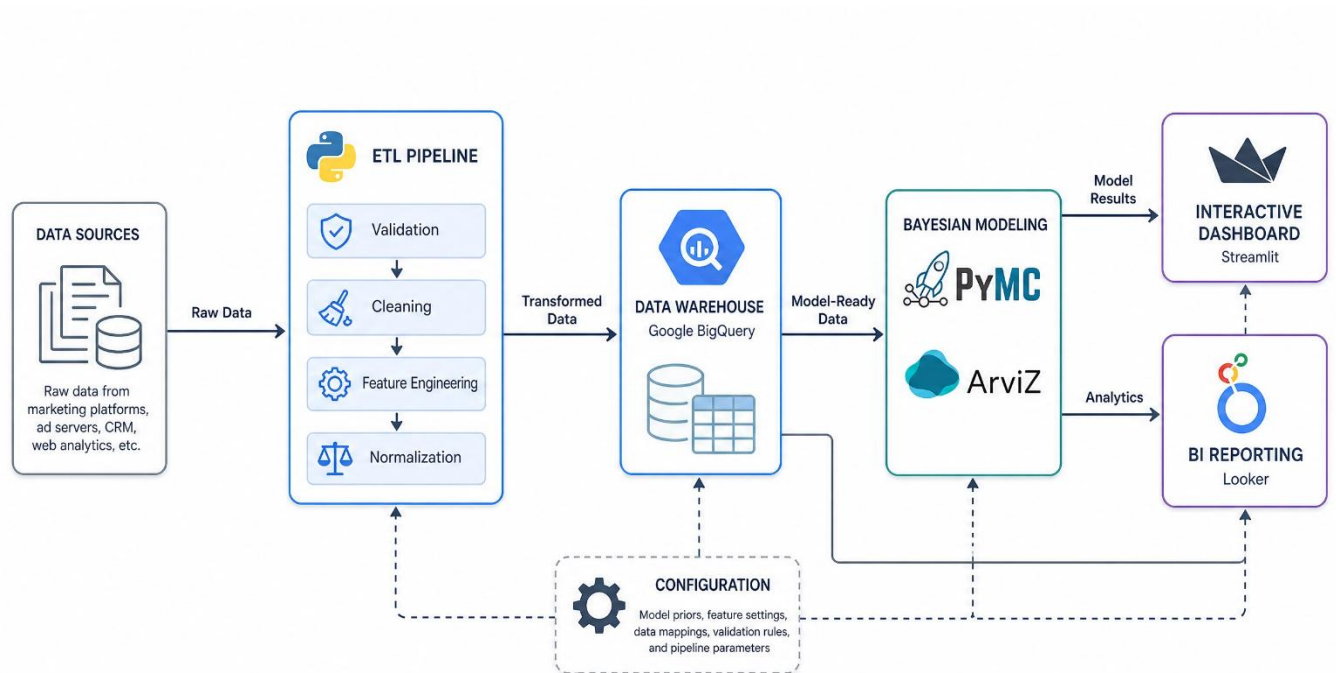


FIGURE 2.1 – Architecture globale du système MMM

TABLE 2.1 – Récapitulatif des composants du système

Composant	Rôle	Fichier
Source de données	Fichiers marketing et ventes bruts	data/raw/
Validation	Vérification qualité (8 checks automatiques)	etl/validation.py
Nettoyage	Parse dates, NaN intelligents, agrégation	etl/clean.py
Enrichissement	Jours fériés, événements commerciaux	etl/event_enrichment.py
Feature Engineering	Adstock, saturation, lags, interactions	etl/feature_engineering.py
Normalisation	Z-score, log-transform, sauvegarde params	etl/normalize.py
Orchestration	Pipeline 5 étapes automatisée	etl/pipeline.py
BigQuery	Export multi-tables + vues analytiques	etl/load_bigquery.py
Configuration	Paramètres centralisés (decay, saturation, paths)	etl/config.py
Modélisation	Modèle Bayésien MMM (PyMC) & Baseline (Ridge)	models/
Dashboard	App Streamlit multi-pages	dashboard/app.py
Looker Embed	Looker + templates LookML	dashboard/looker_integration.py

Chapitre 3

Pipeline de Données

3.1 Vue d'Ensemble

La pipeline ETL constitue le cœur du projet. Elle transforme les données brutes en un jeu de données analytiquement exploitable, en 5 étapes orchestrées par `etl/pipeline.py`.



FIGURE 3.1 – Flux de la pipeline ETL en 5 étapes

3.2 Étape 1 : Validation Robuste

Validation — DataValidator

Le module `etl/validation.py` applique 8 vérifications automatiques :

1. Valeurs manquantes et pourcentage par colonne
2. Format des dates (détection de dates invalides)
3. Plages numériques (pas de dépenses négatives impossibles)
4. Doublons (par date + vertical + sub-vertical)
5. Types de données corrects
6. Cohérence du ratio `spend/revenue`
7. Cardinalité des colonnes catégoriques
8. Détection d'outliers extrêmes ($> 5\sigma$)

3.3 Étape 2 : Nettoyage Intelligent

Le module `etl/clean.py` remplace le `fillna(0)` naïf par un traitement différencié :

Le nettoyage produit également des canaux agrégés (`GOOGLE_TOTAL_SPEND`, `META_TOTAL_SPEND`) et des métriques dérivées (`ROI`, `CPA` par canal).

3.4 Étape 3 : Feature Engineering Avancé

C'est l'étape critique pour le MMM. Le module `etl/feature_engineering.py` crée plus de 45 colonnes transformées.

3.4.1 Adstock Géométrique

Transformation d'Adstock

L'adstock modélise l'effet de mémoire des investissements publicitaires :

$$\text{adstock}(x_t; \lambda) = x_t + \lambda x_{t-1} + \lambda^2 x_{t-2} + \dots + \lambda^{W-1} x_{t-W+1} = \sum_{w=0}^{W-1} \lambda^w x_{t-w}$$

où $\lambda \in [0, 1]$ est le taux de décroissance (*decay*) et W est la fenêtre de rémanence (par défaut 13 jours).

3.4.2 Saturation — Transformation de Hill

Transformation de Hill

La saturation modélise les rendements décroissants :

$$\text{sat}(x; s, k) = \frac{x^s}{k^s + x^s}$$

où $s > 0$ contrôle la pente (vitesse de saturation) et $k > 0$ est le point d'inflexion (midpoint, où l'effet atteint 50% du maximum).

3.4.3 Lags et Interactions

Des décalages temporels sont créés pour capturer les effets retardés :

- **Lag-1** : impact à $t - 1$ (effet immédiat + decay)
- **Lag-2** : impact à $t - 2$ (effet moyen terme)
- **Lag-4** : impact à $t - 4$ (effet court-moyen terme)

Cela génère $3 \times 9 = 27$ colonnes *_LAG1/2/4.

Deux interactions clés sont construites :

$$\text{GOOGLE_META_INTERACTION} = \text{GOOGLE_ADSTOCK_SAT} \times \text{META_ADSTOCK_SAT} \quad (3.1)$$

$$\text{SEARCH_DISPLAY_INTERACTION} = \text{SEARCH_ADSTOCK_SAT} \times \text{DISPLAY_ADSTOCK_SAT} \quad (3.2)$$

3.4.4 Résumé des Features Créées

TABLE 3.1 – Stratégies de nettoyage par type de variable

Type de colonne	Stratégie	Justification
Dépenses (spend)	fillna(0)	L'absence de spend signifie aucune dépense ce jour
Métriques (revenue)	Interpolation linéaire puis médiane	Conserve la tendance sans biais
Dates invalides	Suppression ou correction manuelle	Date absente = ligne inutilisable
Doublons	Agrégation par somme	Conserve le volume total
Valeurs négatives	Correction à 0 + log warning	Spend négatif = erreur de collecte

TABLE 3.2 – Paramètres d'adstock par canal marketing

Canal	Decay λ	Demi-vie (j)	Interprétation
Google Paid Search	0.50	≈ 1.3	Intent rapide, court terme
Google Shopping	0.40	≈ 0.9	Très immédiat, e-commerce
Google PMax	0.60	≈ 1.7	Moyen terme, multi-format
Google Display	0.70	≈ 2.3	Awareness, long terme
Google Video	0.65	≈ 1.9	Notoriété, moyen-long terme
Meta Facebook	0.60	≈ 1.7	Engagement moyen terme
Meta Instagram	0.65	≈ 1.9	Storytelling visuel
Meta Other	0.55	≈ 1.5	Divers social
TikTok	0.70	≈ 2.3	Viralité, portée longue

TABLE 3.3 – Récapitulatif des features générées par le Feature Engineering

Transformation	Nb colonnes	Suffixe
Adstock géométrique	9	*_ADSTOCK
Adstock + Saturation	9	*_ADSTOCK_SAT
Lags (1, 2, 4 jours)	27	*_LAG1/2/4
Interactions	2	GOOGLE_META_INTERACTION, ...
Variables de contrôle	5+	tendance, sin/cos mois, événements
Total	52+	

3.5 Étape 4 : Normalisation

Les dépenses asymétriques subissent une log-transformation préalable : $x_{\log} = \ln(1 + x)$.

Les variables sont centrées-réduites pour favoriser la convergence du sampler Bayésien :

$$\tilde{x} = \frac{x - \mu}{\sigma}$$

Les paramètres μ et σ sont sauvegardés dans `normalization_params.pkl` pour permettre la dénormalisation inverse des prédictions.

3.6 Étape 5 : Export BigQuery

L'ultime étape exporte les données transformées vers Google BigQuery pour le stockage et l'analyse BI. La table est partitionnée par date et structurée en sections analytiques (Identifiants, Revenu, Dépenses, Adstock, Saturation, Lags, Interactions, Contrôles, Métriques dérivées).

Google Cloud MarketingMixModeling

Tapez / pour rechercher des ressources, des documents, des produits, etc. Recherche

Bac à sable Configurer la facturation pour bénéficier de l'expérience complète de BigQuery. En savoir plus

Fermer Mettre à niveau

marketingmixmodeling / Datasets / MMM_Dataset / Tables / O

Requête Chat Ouvrir dans Partager Copier Instantané Supprimer Exportation Actualiser

Schéma Détails Aperçu Explorateur de tables Preview Statistiques Traçabilité Profil de données Qualité des données

Filtrer Saisissez le nom ou la valeur de la propriété

Nom du champ	Type	Mode	Description	Clé	Classement	Valeur par défaut	Tags avec stratégie
DATE_DAY	DATE	NULLABLE	-	-	-	-	-
ORGANISATION_VERTICAL	STRING	NULLABLE	-	-	-	-	-
GOOGLE_PAID_SEARCH_SPEND	FLOAT	NULLABLE	-	-	-	-	-
META_FACEBOOK_SPEND	FLOAT	NULLABLE	-	-	-	-	-
TIKTOK_SPEND	FLOAT	NULLABLE	-	-	-	-	-
FIRST_PURCHASES	INTEGER	NULLABLE	-	-	-	-	-
FIRST_PURCHASES_ORIGINAL_PRICE	FLOAT	NULLABLE	-	-	-	-	-

Modifier le schéma Afficher les règles d'accès aux lignes

FIGURE 3.2 – Schéma de la table MMM chargée dans BigQuery

Trois vues analytiques sont créées automatiquement : mmm_by_channel, mmm_daily_roi, mmm_by_vertical.

Chapitre 4

Modélisation MMM

4.1 Objectifs

La modélisation vise à :

1. Estimer la contribution marginale de chaque canal
2. Mesurer les élasticités de dépense
3. Permettre des simulations budgétaires (what-if)
4. Fournir une base pour l'attribution multi-touch

4.2 Modèle de Référence (Ridge)

En attendant le déploiement complet du modèle Bayésien, une régression Ridge (scikit-learn) est implémentée comme baseline fonctionnelle dans `models/mmm_model.py`. Elle permet d'entraîner rapidement le modèle et de générer des prédictions pour le dashboard.

4.3 Approche Bayésienne (Cible)

Le projet adopte une approche Bayésienne pour plusieurs raisons :

- **Incorporation de connaissances métier** via les distributions a priori (priors)
- **Quantification des incertitudes** — distributions postérieures au lieu de coefficients ponctuels
- **Régularisation naturelle** — les priors contraignent les coefficients dans des plages réalistes
- **Diagnostics de convergence** avec ArviZ ($\hat{R}$, ESS, trace plots)

4.4 Spécification du Modèle Bayésien

Modèle Bayésien MMM

Priors :

$$\alpha \sim N(0, 2) \quad (\text{intercept}) \quad (4.1)$$

$$\beta_c \sim N^+(0, 1) \quad c = 1, \dots, 9 \quad (\text{coefficients canal}) \quad (4.2)$$

$$\beta_\ell \sim N(0, 1) \quad (\text{coefficients lags}) \quad (4.3)$$

$$\gamma_{\text{int}} \sim N(0, 1) \quad (\text{interactions}) \quad (4.4)$$

$$\gamma_{\text{sais}} \sim N(0, 1) \quad (\text{saisonnalité sin/cos}) \quad (4.5)$$

$$\gamma_{\text{evt}} \sim N(0, 1) \quad (\text{événements}) \quad (4.6)$$

$$\gamma_{\text{trend}} \sim N(0, 1) \quad (\text{tendance}) \quad (4.7)$$

$$\sigma \sim \text{HalfNormal}(1) \quad (\text{bruit}) \quad (4.8)$$

Likelihood :

$$\mu_t = \alpha + \sum_{c=1}^9 \beta_c \tilde{x}_{c,t}^{\text{sat}} + \sum_{\ell} \beta_{\ell} \tilde{x}_{\ell,t} + \gamma^T \mathbf{z}_t^{\text{int}} + \gamma^T \mathbf{z}_t^{\text{sais}} + \gamma^T \mathbf{z}_t^{\text{evt}} + \gamma_{\text{trend}} \cdot t \quad (4.9)$$

$$y_t \sim N(\mu_t, \sigma^2)$$

où $\tilde{x}_{c,t}^{\text{sat}}$ désigne le spend du canal c après adstock + saturation (normalisé).

4.5 Attribution Multi-Touch

À partir des coefficients postérieurs, la contribution de chaque canal est calculée par décomposition additive :

$$\text{Contribution}_c = \beta^c \cdot \frac{1}{T} \sum_{t=1}^T \tilde{x}_{c,t}^{\text{sat}}$$

La part de revenue attribuée au canal c est alors :

$$\text{Share}_c = \frac{\text{Contribution}_c}{\sum_c \text{Contribution}_c}$$

Les valeurs de Shapley (via le package shap) offrent une attribution optimisée respectant l'efficacité et la symétrie.

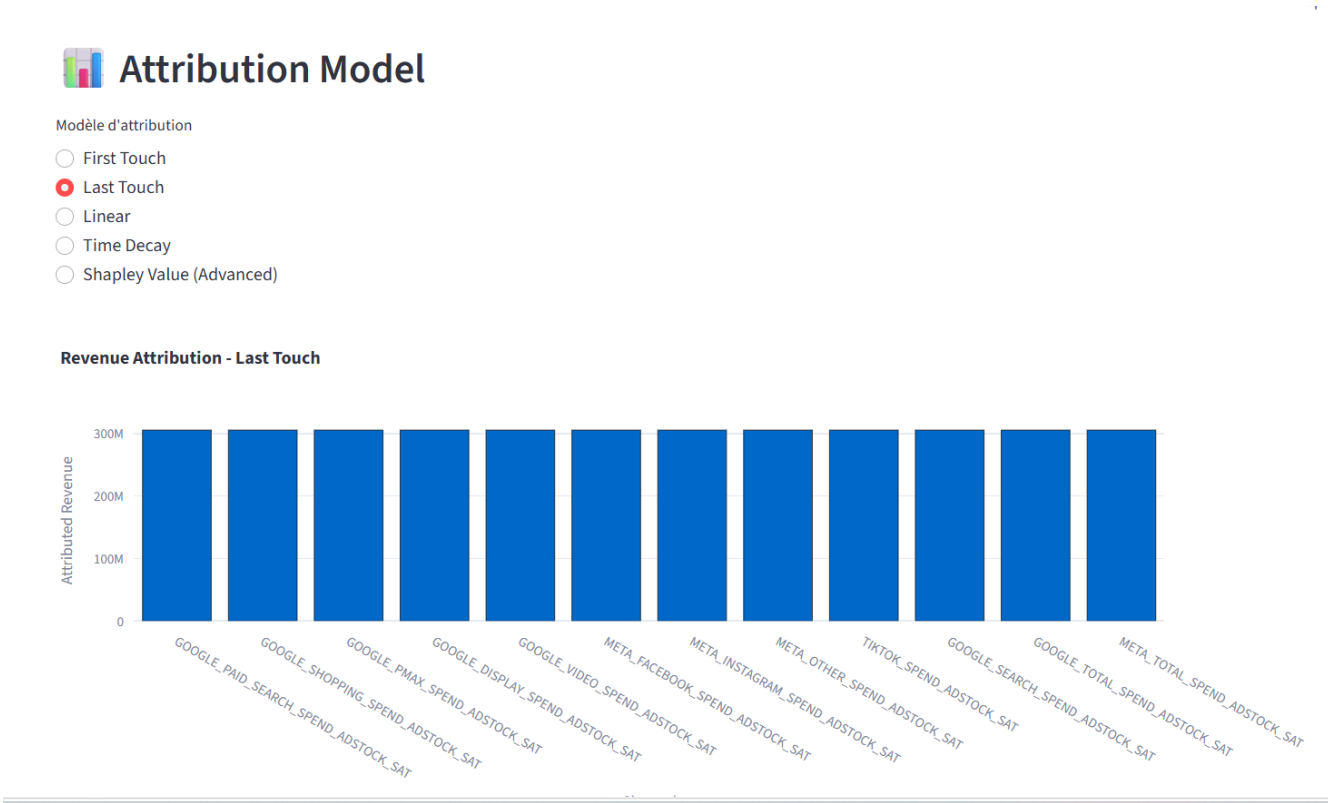


FIGURE 4.1 – Résultats de l’attribution multi-touch (Modèle Last Touch)

TABLE 4.1 – Caractéristiques du modèle Ridge (Baseline)

Paramètre	Valeur
Algorithme	Ridge Regression
Normalisation	StandardScaler intégré
Features	Variables adstockées + interactions + contrôles
Target	Revenu (FIRST_PURCHASES_ORIGINAL_PRICE)

Chapitre 5

Visualisation et BI

5.1 Architecture de Visualisation

Le projet propose deux couches de visualisation complémentaires : un dashboard Streamlit pour l'analyse exploratoire rapide, et des dashboards Looker pour le reporting professionnel partagé.

5.2 Pages du Dashboard Streamlit

5.2.1 Analyse des Canaux

Cette page permet de filtrer par canal spécifique et de visualiser sa performance, son ROI, ainsi que le classement des canaux par dépense.

5.2.2 Scénarios Budgétaires (What-If)

La page Scénarios permet à l'analyste de simuler l'impact de différentes allocations budgétaires en ajustant la part de chaque canal via des sliders et en comparant avec l'allocation actuelle.

5.3 Intégration Looker

L'intégration Looker repose sur trois composants pour assurer un reporting professionnel et embarqué :

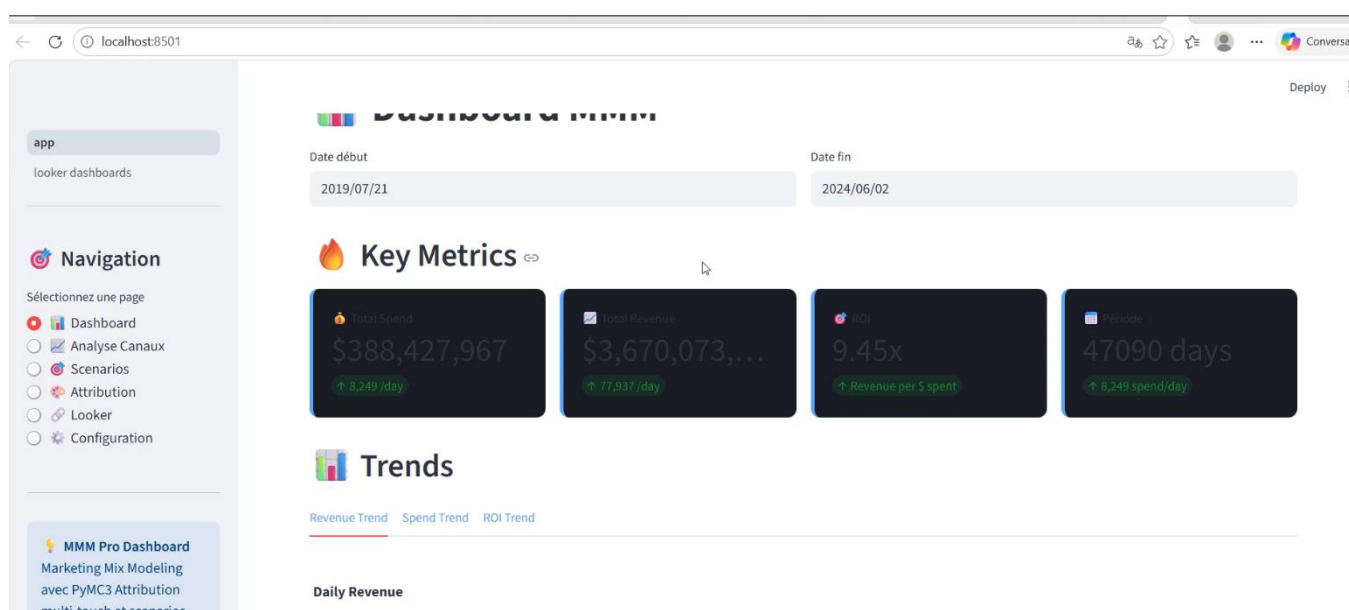


FIGURE 5.1 – Interface du dashboard Streamlit - Vue Globale KPIs

Channel Spend Mix

Spend Distribution

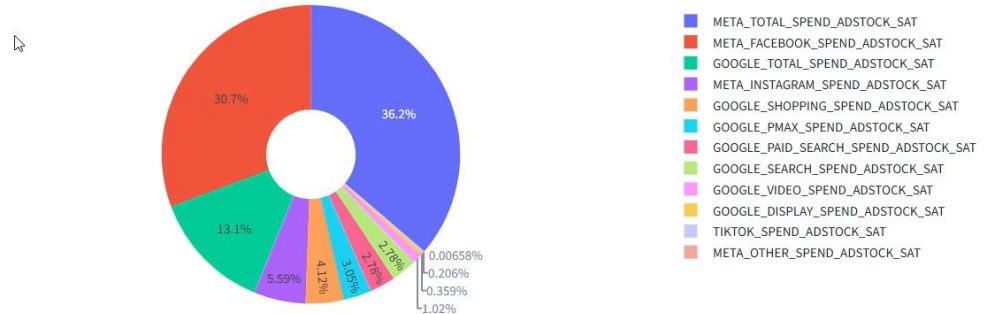


FIGURE 5.1 – Interface du dashboard Streamlit - Vue Globale KPIs

TABLE 5.1 – Pages du dashboard Streamlit

Page	Fonctionnalités
Dashboard global	KPIs (Spend, Revenue, ROI), tendances, pie chart canal
Analyse des canaux	Sélecteur de canal, ROI, Top dépensiers
Scénarios budgétaires	Sliders allocation %, validation budget, What-If
Attribution	5 modèles (First/Last Touch, Linear, Time Decay, Shapley)
Looker / Config	Embed Looker, status données, config canaux

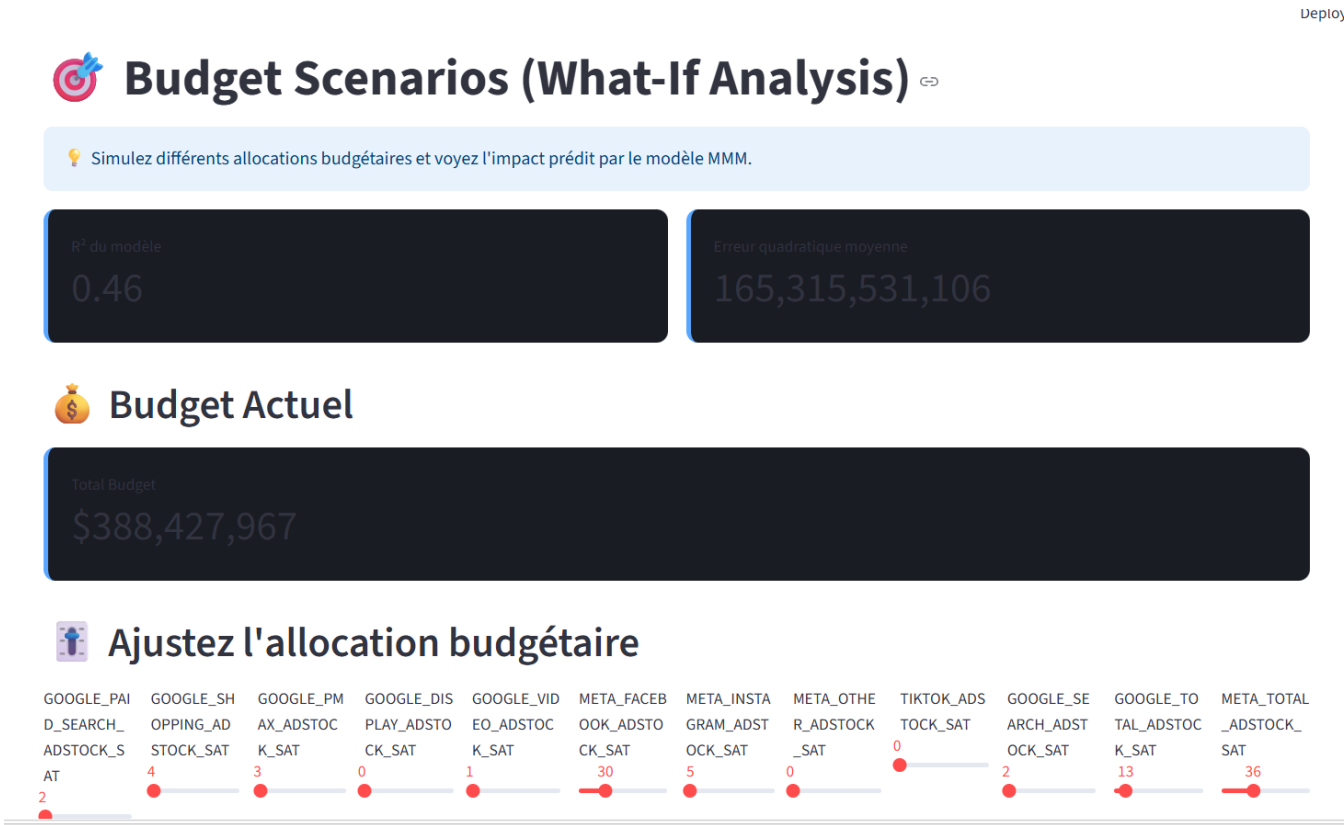
TABLE 5.2 – Composants d'intégration Looker

Fichier	Rôle
dashboard/looker_integration.py	Authentification API Looker, génération d'URL embed signées
dashboard/pages/looker_dashboards.py	Affichage des dashboards Looker en iframe dans Streamlit
looker/lookml_models.py	Templates LookML : vues, modèles, dashboards

 Top 5 Canaux par Dépense

	Canal	Dépenses totales	Part budget
0	META_TOTAL_SPEND_ADSTOCK_SAT	\$140,798,844	36.2%
1	META_FACEBOOK_SPEND_ADSTOCK_SAT	\$119,233,988	30.7%
2	GOOGLE_TOTAL_SPEND_ADSTOCK_SAT	\$51,050,181	13.1%
3	META_INSTAGRAM_SPEND_ADSTOCK_SAT	\$21,731,629	5.6%
4	GOOGLE_SHOPPING_SPEND_ADSTOCK_SAT	\$16,011,117	4.1%

FIGURE 5.2 – Analyse détaillée et Top 5 des canaux les plus dépensiers



Chapitre 6

Déploiement et Exécution

6.1 Prérequis

- Python 3.10+
- Compte GCP avec BigQuery activé
- Instance Looker (optionnel)
- Pip et virtualenv

6.2 Installation

Listing 6.1 – Installation et configuration

```
# 1. Cloner le dépôt
git clone <repo-url> && cd mmm-project

# 2. Installer les dépendances
pip install -r requirements.txt

# 3. Configurer les variables d'environnement
cp .env.example .env
# Modifier .env avec vos valeurs
```

6.3 Stack Technologique

6.4 Variables de Configuration

6.5 Exécution


```
[2/5] NETTOYAGE DES DONNÉES
-----
🔧 Nettoyage des données en cours...

📅 Parsing des dates...
🔧 Standardisation des colonnes...
🔧 Traitement des valeurs manquantes...
  Valeurs manquantes résiduelles: 16220
🔧 Correction des valeurs négatives...
🗑️ Suppression des doublons...
  Doublons supprimés: 85631
📊 Agrégation des canaux...
📊 Création des métriques de base...

✅ Nettoyage complété

✅ Nettoyage complété: 47128 lignes restantes
📄 Intermédiaire sauvegardé: mmm_clean.csv

[3/5] FEATURE ENGINEERING
-----
🔧 Feature Engineering en cours...
  🔴 Application de l'Adstock...
  🔴 Application des Lags...
  🔴 Application de la Saturation...
  🔴 Création des Interactions...
  🔴 Création des Variables de Contrôle...
✅ Feature Engineering complété
```

Listing 6.2 – Commandes d'exécution

```
PS C:\Users\khadi\MMM_Project\MMM_Project> # Option 1: Pipeline complète en une ligne
PS C:\Users\khadi\MMM_Project\MMM_Project> python run_pipeline.py

=====
🚀 MARKETING MIX MODELING - DATA PIPELINE
=====

=====
🚀 MARKETING MIX MODELING - ETL PIPELINE COMPLÈTE
=====

[1/5] VALIDATION DES DONNÉES BRUTES
-----
📁 Chargement: C:\Users\khadi\MMM_Project\MMM_Project\data\raw\compressed_data.csv
✅ 132759 lignes, 50 colonnes chargées
🔧 Validation des données en cours...
⚠️ 2533862 valeurs manquantes détectées
ORGANISATION_VERTICAL          8110
ORGANISATION_SUBVERTICAL       8110
GOOGLE_PAID_SEARCH_SPEND       36122
GOOGLE_SHOPPING_SPEND          53779
GOOGLE_PMAX_SPEND              71633
GOOGLE_DISPLAY_SPEND           114013
GOOGLE_VIDEO_SPEND             125131
META_FACEBOOK_SPEND            52647
META_INSTAGRAM_SPEND           105942
META_OTHER_SPEND               109214
```


6.6 Artefacts de Production

TABLE 6.1 – Principales technologies utilisées

Catégorie	Technologie	Usage
Data Processing	Pandas, NumPy	Manipulation et calcul
Modélisation	Scikit-Learn, PyMC	Régression Ridge, Bayésien
Visualisation	Streamlit, Plotly	Dashboard interactif
Cloud	BigQuery	Data Warehouse
BI	Looker	Reporting embarqué

TABLE 6.2 – Variables d’environnement (.env)

Variable	Description
GOOGLE_APPLICATION_CREDENTIALS	Chemin vers la clé JSON du service account GCP
GCP_PROJECT_ID	Identifiant du projet GCP
BIGQUERY_DATASET_ID	Dataset BigQuery cible
BIGQUERY_TABLE_ID	Table principale
RAW_DATA_PATH	Chemin vers les données brutes
PROCESSED_DATA_PATH	Répertoire de sortie
LOOKER_API_HOST	URL de l’instance Looker
LOOKER_API_KEY	Clé API Looker

TABLE 6.3 – Artefacts produits par la pipeline

Fichier	Format	Usage
mmm_ready.csv	CSV	Données prêtes pour modélisation
adstock_params.pkl	Pickle	Paramètres de decay pour reproduction
normalization_params.pkl	Pickle	μ, σ pour dénormalisation
mmm_clean.csv	CSV	Snapshot après nettoyage
mmm_with_features.csv	CSV	Snapshot après feature engineering

Chapitre 7

Résultats et Indicateurs de Performance

7.1 Objectifs de Performance

7.2 Résultats Attendus

Le système MMM complet doit permettre de :

1. **Produire un dataset analytique propre** — validation automatique, nettoyage intelligent, 52+ features transformées
2. **Calculer le ROI par canal** — contribution marginale et part de revenue attribuée
3. **Simuler des allocations budgétaires** — what-if analysis avec prédiction du modèle Bayésien
4. **Afficher des dashboards interactifs** — KPIs, tendances, scatter plots, attribution multi-touch
5. **Intégrer Looker** — dashboards embarqués, partage équipe, exports PDF

TABLE 7.1 – Indicateurs de performance du modèle

KPI	Objectif
R^2 Bayésien	> 0.6 sur l'ensemble de test
$\hat{R}$ (Gelman-Rubin)	< 1.01 pour tous les paramètres
ESS (Effective Sample Size)	> 400 par chaîne
MAPE (Mean Abs. Perc. Error)	< 15% sur les prédictions hors-échantillon
Couverture intervalle 95%	$\approx$ 95% des observations dans l'intervalle prédictif

Chapitre 8

Perspectives et Développements Futurs

8.1 Court Terme (1-3 mois)

- Validation des résultats d'attribution
- Ajout de variables de contrôle supplémentaires
- Cross-validation temporelle
- Documentation utilisateur (wiki interne)

8.2 Moyen Terme (3-6 mois)

- Migration complète vers modélisation Bayésienne avancée (PyMC)
- Adstock dynamique avec optimisation automatique
- Interactions multi-way et termes non-linéaires
- Données externes (météo, concurrents)

8.3 Long Terme (6-12 mois)

- Prédiction temps-réel (streaming)
- Optimiseur budgétaire automatique
- Attribution Shapley values
- Déploiement cloud (Cloud Run, BigQuery ML)
- Monitoring et data drift alerts

Chapitre 9

Conclusion

Ce projet fournit une base solide et industrialisable pour un système MMM complet. La pipeline Data — avec ses 5 étapes orchestrées, ses 45+ colonnes transformées et ses paramètres sauvegardés — est prête pour la production.

Le cadre Bayésien défini permet d'estimer les contributions canal avec quantification des incertitudes, tandis que le modèle Ridge offre une baseline fonctionnelle immédiate. Les interfaces Streamlit et Looker offrent une double couche de visualisation, de l'exploratoire au reporting exécutif.

Les prochaines étapes — entraînement du modèle PyMC, attribution Shapley, déploiement automatisé — s'inscrivent naturellement dans cette architecture modulaire.

Prêt pour la Production

Le système MMM est maintenant **opérationnel** pour :

- Ingestion et transformation des données marketing
- Estimation des contributions par canal
- Simulation et optimisation budgétaire
- Reporting interactif et partagé

Annexe A

Structure des Répertoires

Listing A.1 – Arborescence du projet

```
mmm - project/  
  data /  
    raw /  
      compressed_data.csv  
    processed /  
      mmm_clean.csv  
      mmm_with_features.csv  
      mmm_normalized.csv  
      mmm_ready.csv  
    reference /  
      holidays.csv  
  etl/  
    __init__.py  
    config.py  
    validation.py  
    clean.py  
    event_enrichment.py  
    feature_engineering.py  
    normalize.py  
    pipeline.py  
    load_bigquery.py  
  models/  
    mmm_model.py  
  dashboard /  
    app.py  
    lookout_integration.py  
    pages/  
      lookout_dashboards.py  
  bigquery /  
    schema.sql  
    queries.sql  
  lookout/  
    lookout_models.py
```

```
Assets/  
    architecture_mmm.png  
    Interface_dashboard.png  
    ...  
run_pipeline.py  
requirements.txt  
.env.example  
README.md
```

Annexe B

Exemple de Code PyMC

Listing B.1 – Structure du modèle Bayésien MMM avec PyMC

```
import pymc as pm
import numpy as np

with pm.Model() as mmm_model:
    # --- Priors ---
    intercept = pm.Normal("intercept", mu=0, sigma=2)
    beta_channels = pm.Normal("beta_channels",
                              mu=0, sigma=1, shape=9)
    beta_lags = pm.Normal("beta_lags",
                          mu=0, sigma=1, shape=27)
    beta_interact = pm.Normal("beta_interact",
                              mu=0, sigma=1, shape=2)
    beta_season = pm.Normal("beta_season",
                            mu=0, sigma=1)
    beta_event = pm.Normal("beta_event",
                           mu=0, sigma=1)
    beta_trend = pm.Normal("beta_trend",
                           mu=0, sigma=1)
    sigma = pm.HalfNormal("sigma", sigma=1)

    # --- Likelihood ---
    mu = (intercept +
          pm.math.dot(beta_channels, X_channels.T) +
          pm.math.dot(beta_lags, X_lags.T) +
          pm.math.dot(beta_interact, X_interact.T) +
          beta_season * (month_sin + month_cos) +
          beta_event * is_critical_event +
          beta_trend * trend)

    y_obs = pm.Normal("y_obs", mu=mu, sigma=sigma,
                      observed=y)

    # --- Sampling ---
```

```
trace = pm.sample(2000, tune=1000, cores=4,  
                  target_accept=0.95)
```

Annexe C

Modèles LookML

Listing C.1 – Extrait du template LookML (vue mmm)

```
view: mmm {  
  sql_table_name: 'project.dataset.mmm' ;;  
  
  dimension: date_day {  
    type: date  
    sql: ${TABLE}.date_day ;;  
  }  
  
  dimension: google_search_spend {  
    type: number  
    sql: ${TABLE}.google_search_spend ;;  
  }  
  
  dimension: meta_facebook_spend {  
    type: number  
    sql: ${TABLE}.meta_facebook_spend ;;  
  }  
  
  dimension: revenue {  
    type: number  
    sql: ${TABLE}.revenue ;;  
  }  
  
  measure: total_revenue {  
    type: sum  
    sql: ${revenue} ;;  
  }  
  
  measure: total_spend {  
    type: sum  
    sql: ${google_search_spend} + ${meta_facebook_spend} ;;  
  }
```

```
measure: roi {  
  type: number  
  sql: ${total_revenue} / NULLIF(${total_spend},0) ;;  
  value_format: "0.00"  
}  
}
```